WEBMAPPING

OPENLAYERS

WEBMAPPING

Openlayers

Deux librairies principales pour du webmapping 2D open source

Leaflet	OpenLayers
Simple, mais extensible par des plugins	Beaucoup de fonctionnalités de base + plugins
Grande communauté, beaucoup d'exemples	Très à jour sur les nouveaux standards
Projection Suisse peu supportée	Très présente sur le marché Suisse

WEBMAPPING

Openlayers

OpenLayers est très populaire en Suisse mais aussi ailleurs dans le monde, il sert de base à de nombreux géoportails.

- GeoAdmin : https://map.geo.admin.ch
- SuisseMobile : https://map.wanderland.ch
- Luxembourg : https://map.geoportail.lu
- EPFL : https://plan.epfl.ch
- SITN : https://sitn.ne.ch
- Transports publics en temps réel : https://tracker.geops.ch
- Saint-Pierre de la Réunion : https://geo.saintpierre.re
- Région de Nyon : https://map.cartolacote.ch

WEBMAPPING

Openlayers

OpenLayers est très bien documenté

- Site officiel : https://openlayers.org/
- Quickstart : https://openlayers.org/doc/quickstart.html
- API complète : https://openlayers.org/en/latest/apidoc/
- La longue liste d'exemples : https://openlayers.org/en/latest/examples/

Nous allons voir les bases d'OpenLayers, mais il est attendu que vous appreniez à utiliser ces ressources.

Les exemples sont le meilleure moyen d'apprendre rapidement, OpenLayers en contient un vaste catalogue.

WEBMAPPING

Openlayers

Comment ça marche ?

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Quick Start</title>
</head>
<body>
    <div id="map"></div>
    <script type="module" src="./main.js"></script>
</body>
</html>
```

OpenLayers a besoin d'un `<div>` dans lequel sera placé la carte, il a ici l'id `map`.

Le script principal est ensuite inclus, comme dans un projet Vite de base.

WEBMAPPING

Openlayers

Comment ça marche ?

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Quick Start</title>
</head>
<body>
    <div id="map"></div>
    <script type="module" src="./main.js"></script>
</body>
</html>
```

OpenLayers a besoin d'un `<div>` dans lequel sera placé la carte, il a ici l'id `map`.

Le script principal est ensuite inclus, comme dans un projet Vite de base.

WEBMAPPING

Openlayers

La Map

C'est l'objet de base, il représente la carte affichée à l'écran.

Propriétés :

- `target` : id de l'élément dans lequel créer la `Map`
- `view` : Objet `View` qui décrit la vue à afficher dans la carte
- `layers` : Tableau d'objets `Layers` qui décrivent les couches présentes sur la carte

De manière facultative

- `controls` : Tableau d'objets `Control` décrivant les interactions avec la carte (zoom, rotation, plein écran, etc)
- `interactions` : Tableau d'actions pour lesquelles la carte doit réagir (clic, double clic, drag-drop, etc)

D'autres propriétés existent, voir https://openlayers.org/en/latest/apidoc/module-ol_Map-Map.html

WEBMAPPING

Openlayers

La Map

```
import './style.css';
import Map from 'ol/Map.js';

const map = new Map({
    target: 'map',          // 'target' est l'id de la div à utiliser comme carte
    view: /* A venir */,
    layers: /* A venir */
});
```

Dans OpenLayers, les paramètres sont toujours passés à l'aide d'un objet. On écrira :

```
new Map({target:'map', view:undefined, layers:undefined});
```

Et non pas

```
new Map('map', undefined, undefined);
```

WEBMAPPING

Openlayers

La View

Objet qui décrit la résolution, la rotation, la position, la projection, etc. de la `Map`

Propriétés :

- `center` : tableau contenant les coordonnées du centre de la carte
- `zoom` : nombre décrivant le niveau de zoom

Autres propriétés

- `resolution` : alternative à `zoom`, qui décrit la résolution de la carte
- `projection` : la projection de la vue

D'autres propriétés existent, voir https://openlayers.org/en/latest/apidoc/module-ol_View-View.html

WEBMAPPING

Openlayers

La View

```
import './style.css';
import Map from 'ol/Map.js';
import View from 'ol/View.js';

const map = new Map({
    target: 'map',
    view: new View({        // La View indique comment la Map doit être affichée
        center: [0, 0],     // Ici centrée en 0;0
        zoom: 2,            // Et avec un zoom à 2
    }),
    layers: /* A venir */
});
```

On remarque que `Map` et `View` prennent en paramètre un objet qui contient toutes les valeurs des paramètres.

WEBMAPPING

Openlayers

Les Layers

Il existe plusieurs types de layers (module `ol/layer`), mais ils peuvent être divisés en 2 catégories :

1. Raster (par ex: `TileLayer`, `ImageLayer`)
2. Vectoriel (par ex: `VectorLayer`, `VectorTileLayer`)

La `source` de la donnée (module `ol/source`) est une propriété d'un `layer` et existe aussi en 2 catégories :

1. Raster (par ex: `TileSource`, `ImageSource`)
2. Vectoriel (par ex: `VectorSource`, `VectorTile`)

Un exemple de `TileSource` est un objet `OSM` qui indique une couche OpenStreetMap.

WEBMAPPING

Openlayers

Code complet de *script.js*

```
import './style.css';
import Map from 'ol/Map.js';
import OSM from 'ol/source/OSM.js';
import TileLayer from 'ol/layer/Tile.js';
import View from 'ol/View.js';
const map = new Map({
    target: 'map',
    view: new View({
        center: [0, 0],
        zoom: 2,
    }),
    layers: [               // Liste des layers à afficher
        new TileLayer({     // Dans ce cas il y a un layer tuilé, OpenStreetMap (OSM)
            source: new OSM(),
        }),
    ],
```

WEBMAPPING

Openlayers

Les Contrôles
Les contrôles sont des éléments permettant de manipuler la carte ou d'afficher une information.
Par défaut, `Map` en charge 3:

- `Zoom` : Permet le zoom sur le carte
- `Rotate` : Orientation (apparaît dès que la carte est tournée)
- `Attribution` : Copyright des données

Il en existe d'autres

- Barre d'échelle : `ScaleLine`
- Carte d'aperçu : `OverviewMap`
- Position curseur : `MousePosition`
- Plein écran : `FullScreen`
- Zoom sur étendue max : `ZoomToExtent`
- Curseur de zoom : `ZoomSlider`

Exemples : https://openlayers.org/en/latest/examples/?q=control

WEBMAPPING

Openlayers

La projection suisse EPSG:2056

Par défaut, OpenLayers ne connaît pas la projection suisse. Il faut la lui définir.

```
import proj4 from "proj4";
import { register } from "ol/proj/proj4";

proj4.defs(
    "EPSG:2056",
    "+proj=somerc +lat_0=46.95240555555556 +lon_0=7.439583333333333 +k_0=1 +x_0=2600000" +
    " +y_0=1200000 +ellps=bessel +towgs84=674.374,15.056,405.346,0,0,0,0 +units=m +no_defs",
);
register(proj4);
```

Ces lignes sont à ajouter au début de votre script, avant tout appel à OpenLayers